

Artifact:

Context: I needed to strip dollar signs from the Amount column in a transaction CSV so amounts could be used as numbers in calculations.

My Prompt: "Help me clean dollar sign amounts from a pandas column for my finance app"

AI Response Summary: The AI provided a basic `.str.replace('$', '')` followed by `.astype(float)` — which works only for simple positive dollar amounts with no edge cases.

My Critique/Improvement: The initial response broke on negative amounts like `-$15.00` (minus sign before the dollar sign) and crashed on values with spaces or commas. I asked the AI to use regex and handle unconvertible values with `errors='coerce'` instead of crashing.



Result: The improved version uses a single regex `r'[\$,\\s]'` to strip all problematic characters in one pass, then `pd.to_numeric(..., errors='coerce')` converts safely and `dropna` removes any rows that couldn't be parsed — making the function robust for real-world messy data.

Reflection: Testing with the Xbox Store refund row (`-$15.00` on 2026-03-20) immediately revealed the edge case. The regex approach handles all currency formatting variations at once and is far more production-ready than a simple replace.

Entry 2 – Loading CSV Files for Gradio



Artifact:



Write a function to load a CSV file for my finance assistant. It should work with file paths

Python



```
def load_and_clean(file_path):  
    df = pd.read_csv(file_path)  
    df['Amount_Clean'] = pd.to_numeric(  
        df['Amount'].str.replace('$', ''), e  
    )  
    return df
```



This only works with file paths but in Gradio the file comes as an object with a .name attribute, not a plain string. I also want to pass a raw CSV string for testing. Can you handle all three cases and validate that the required columns exist before continuing?

Python



```
def load_and_clean(file_or_string):  
    try:  
        if hasattr(file_or_string, 'name'):  
            df = pd.read_csv(file_or_string.name)  
        elif isinstance(file_or_string, str):  
            df = pd.read_csv(io.StringIO(file_or_string))  
        else:  
            df = pd.read_csv(file_or_string)  
    except Exception as e:  
        raise ValueError(f'Could not read file: {e}')  
  
    required = ['Date', 'Amount', 'Category']  
    missing = [c for c in required if c not in df.columns]  
    if missing:  
        raise ValueError(f'Missing columns: {missing}')  
    ...
```





Context: I needed a single function that could accept a CSV from three different sources — a Gradio file upload, a raw string for testing, or a plain file path.

My Prompt: "Write a function to load a CSV file for my finance assistant. It should work with file paths"

AI Response Summary: The AI returned a simple `pd.read_csv(file_path)` function that only accepted a plain string file path, with no handling for Gradio's file object format or raw CSV strings.

My Critique/Improvement: In Gradio, uploaded files are objects with a `.name` attribute, not plain strings — so the function would crash on every real upload. I asked the AI to handle all three input types and validate that required columns exist before continuing.

Result: The improved version uses `hasattr`, `isinstance`, and `io.StringIO` to detect all three input formats, wraps reading in `try/except`, and raises a descriptive `ValueError` listing any missing columns — making the function production-ready.

Reflection: The original only handled file paths. Adding the three-way check and column validation meant the function works in every scenario — Gradio upload, test string, or direct path — without unexpected crashes.

Entry 3 – Spending Analysis Groupby



Artifact:



I have a cleaned DataFrame with Amount_Clean. Write code to group by Category and calculate totals

Python



```
# Group by category
result = df.groupby('Category')['Amount_Clean'].agg(['sum', 'mean'])
print(result)
```



This gives unreadable column names and does not include a transaction count. It also does not sort by highest spending or add a percentage column. Can you rewrite it with named aggregations, a percentage column, and sort descending by total spending?

Python



```
# Filter to positive spending only – exclude negative
positive = df[df['Amount_Clean'] > 0].copy()
total_spent = positive['Amount_Clean'].sum()

# Named aggregations give readable column names
summary = (
    positive.groupby('Category')['Amount_Clean']
    .agg(Total_Spent='sum', Avg_Transaction='mean')
    .round(2).reset_index()
)

# Add percentage of total spending per category
summary['Percentage'] = (summary['Total_Spent'] / total_spent) * 100
summary = summary.sort_values('Total_Spent', ascending=False)
```



Context: I wanted to group transactions by category and calculate total spending, average transaction amount, count, and percentage of total budget for each category.

My Prompt: "I have a cleaned DataFrame with Amount_Clean. Write code to group by Category and calculate totals"

AI Response Summary: The AI used `df.groupby('Category')['Amount_Clean'].agg({'sum', 'mean'})` which produces unreadable auto-generated column names and includes no transaction count, sorting, or percentage column.

25/05/2026 16:02Developer_Diary.ipynb - Colab

 **My Critique/Improvement:** I asked for named aggregations for readable column names, a transaction count, a percentage column, and descending sort by total spending. I also noted that refund rows needed to be excluded before grouping.

Result: The improved version filters to positive-only rows first, uses named aggregations (`Total_Spent='sum'`, `Avg_Transaction='mean'`, `Count='count'`), adds a `Percentage` column relative to filtered total spending, and sorts descending — ready for use in charts and reports.

Reflection: Named aggregations like `agg(Total_Spent='sum')` are much cleaner than set syntax. Separating refunds before grouping was critical — the Xbox refunds on 2026-03-20 and 2026-04-18 were inflating totals and breaking percentages without this filter.

Entry 4 – Text Bar Chart in Report

Artifact:

I want to add a simple text bar chart to my spending report using the percentage column

Python

```
for index, row in summary.iterrows():
    bar = '#' * row['Percentage']
    print(f"{row['Category']} {bar}")
```

This crashes when Percentage is a float like 38.2 because you cannot multiply a string by a float. The bars are also way too long at 38 characters. Can you fix the type error, scale it so 1 block represents 5 percent, and use the block character instead of # to look more professional?

Python

```
for _, row in summary.iterrows():
    # int() fixes the float crash; dividing by 5
    # max(1,...) ensures even tiny categories have a bar
    bar = chr(9608) * max(1, int(row['Percentage'] / 5))
    print(
        f" {row['Category']:<15} ${row['Total_Spent']}
        f" ({row['Percentage']:>5.1f}%) {bar}"
    )
```

Context: I wanted to add a visual element to the spending report so users could instantly see which category they spent the most on without reading all the numbers.

My Prompt: "I want to add a simple text bar chart to my spending report using the percentage column"



Reflection: The float-to-int crash is easy to miss without testing on real data. Using `max(1, ...)` ensures even small categories like a single Maison de The visit still appear in the chart rather than disappearing entirely.

Entry 5 – Handling Missing Columns Gracefully

Artifact:



If someone uploads a CSV without the right columns, my app crashes with a `KeyError`. How do I give a friendly error?

Python



```
if 'Amount' not in df.columns:
    print('Error: missing Amount column')
```



This only checks for Amount but my app needs Date, Amount, Category, and Description. Also just printing is useless inside a function — I need to raise a `ValueError` so the Gradio wrapper can catch it and show the message to the user. Can you check all four columns at once?

Python



```
required = ['Date', 'Amount', 'Category', 'D  
missing = [c for c in required if c not in d  
if missing:  
    raise ValueError(  
        f'Missing columns: {missing}. '  
        f'Your CSV must have: {required}'  
    )
```



My Prompt: "If someone uploads a CSV without the right columns, my app crashes with a KeyError. How do I give a friendly error?"

25/05/2026 16:02Developer_Diary.ipynb - Colab

 **AI Response Summary:** The AI checked only for the `Amount` column using a single `if 'Amount' not in df.columns: print(...)` — missing three other required columns and using `print` instead of raising an exception.

My Critique/Improvement: I asked for all four required columns to be checked at once using a list comprehension, and for a `ValueError` to be raised (not printed) so the Gradio wrapper can catch it and display the message in the output textbox.

Result: The improved version checks all four columns in one list comprehension, raises a `ValueError` with a message listing both the missing and required columns — giving users an immediately actionable error.

Reflection: Raising `ValueError` instead of just printing means the Gradio wrapper catches it and displays it cleanly in the output textbox. Checking all four columns with a list comprehension is much cleaner than four separate `if` statements.

Entry 6 – FinBot System Prompt Design

Artifact:

Write a system prompt for a finance chatbot

Code

You are a helpful financial assistant.
Answer all finance questions clearly and help users.
Give investment advice when asked.

This is too generic and actually tells it to give investment advice which is dangerous. I need the personality specific to my user someone who games on Xbox, goes to the gym, watches movies at MCine in Moka, and drinks tea at Maison de The. It should never give stock or investment advice. Can you rewrite it with all of that context?

Code

You are FinBot, a friendly personal finance advisor who enjoys Xbox gaming, going to the gym, watching movies at MCine in Moka, and drinking tea at Maison de The in Moka.

Help them understand their spending, budget and save around their lifestyle. Be concise, practical and helpful. Use dollar amounts. Never give stock investment advice. If spending context is provided, reference it.

Context: I needed a system prompt that gives the chatbot a specific finance advisor personality relevant to my user's lifestyle — Xbox gaming, gym, MCine cinema, and Maison de The tea.



My Prompt: "Write a system prompt for a finance chatbot"

AI Response Summary: The AI returned a three-line generic prompt saying "You are a helpful financial assistant" and explicitly told it to give investment advice when asked — the opposite of what a student finance app should do.

My Critique/Improvement: I asked for a personality specific to the user's lifestyle (Xbox, gym, MCine Moka, Maison de The), an explicit instruction to never give stock or investment advice, and a directive to reference spending context when provided.

Result: The improved system prompt names the user's specific habits and locations, restricts advice to budgeting and savings, and instructs the model to use any spending context passed to it — making FinBot's responses personal and safe.

Reflection: The first version was completely generic and encouraged investment advice — the one thing a student finance app should never do. Making the system prompt specific to Xbox, MCine, and Maison de The means FinBot gives much more relevant and personal responses.

Entry 7 – Passing Spending Context to Chatbot



Artifact:

How do I pass the user's spending data into the chatbot so it can give personalised advice?

Python

```
def chat_finbot(user_msg):
    reply = get_response(user_msg, system=SYSTEM_PROMPT)
    return reply
```

"This function never uses the spending data at all. I want to pass the top spending categories as a string and append them to the system prompt so FinBot knows what the user actually spent. Can you add a spending_ctx parameter that gets included in the system prompt only when provided?"

Python

```
def chat_finbot(user_msg, spending_ctx=''):
    system = SYSTEM_PROMPT
    if spending_ctx:
        system += f'\n\n[User current spending: {spending_ctx}]'
    try:
        reply = get_response(user_msg, system=system, chat_history=chat_history)
        chat_history.append((user_msg, reply))
        return reply
    except Exception as e:
        return f'Connection error: {e}'
```

Context: I wanted FinBot to reference the user's actual spending data when answering questions, so the advice is personalised rather than generic.

My Prompt: "How do I pass the user's spending data into the chatbot so it can give personalised advice?"

AI Response Summary: The AI returned a `chat_finbot(user_msg)` function that called `get_response` with only the user message and system prompt — the spending data was never passed in at all.

My Critique/Improvement: I asked for a `spending_ctx` parameter that gets appended to the system prompt when provided, chat history to be recorded, and a try/except to return a friendly connection error message instead of crashing.

Result: The improved version conditionally appends spending context to the system prompt (keeping the user's question natural), logs history, and handles API exceptions gracefully — giving FinBot the data it needs without cluttering the user's message.



Reflection: Appending context to the system prompt rather than the user message keeps the user's question natural while giving FinBot the spending data it needs. Testing with 'Gaming 216.96, *Gym*85.00, Cinema 35.00, *TeaandCafe*32.50' confirmed it gives category-specific advice.

Entry 8 – RAG Knowledge Base Content



Artifact:

Write a short financial tips document I can use for RAG in my finance assistant

Code

Financial Tips:

- Save money
- Don't spend too much
- Have an emergency fund
- Budget your money
- Avoid debt

These tips are way too vague for RAG. When someone asks how do I spend less on tea at Maison de The, the RAG system needs specific content to retrieve from. Can you write detailed sections for Xbox gaming, gym memberships, MCine cinema, Maison de The tea, the 50/30/20 rule, and impulse buying with specific dollar figures and local context?

Code

Gaming Spending

Xbox Game Pass Ultimate is cost-effective if you play more than 10 hours a week. Use the 48-hour wishlist rule before buying. Xbox Sales happen monthly and save 20-60% on titles.

Tea at Maison de The Moka

A daily visit costs \$7.50 to \$9.00. At 3 visits a week, that's \$22.50 to \$27.00. Home loose-leaf tea costs around \$100 per month.

Cinema at MCine

MCine loyalty cards reduce ticket prices significantly. Going on Tuesday or Wednesday saves \$2-3 per ticket.

Gym Membership

Monthly fees total \$720+ per year. Off-peak sessions are cheaper. Block-booking PT sessions gives a discount for future sessions.



Context: I needed to create a detailed financial tips document that the RAG system could retrieve specific answers from when users ask questions about their spending habits.

My Prompt: "Write a short financial tips document I can use for RAG in my finance assistant"

AI Response Summary: The AI returned five one-line bullet points ("Save money", "Don't spend too much", "Have an emergency fund") — far too vague for a RAG system to retrieve anything specific or useful from.

My Critique/Improvement: I asked for detailed, location-specific sections covering Xbox gaming (with the 48-hour wishlist rule), gym memberships (off-peak savings), MCine cinema (loyalty cards, weekday pricing), Maison de The tea (daily cost vs home brewing), the 50/30/20 rule, and impulse buying — all with specific dollar figures.

Result: The improved document contains named sections with actionable figures (e.g. "A daily visit costs 7.50to9.00") that the RAG system can retrieve precisely — the tea section triggers on cafe questions and the gaming section on Xbox questions.

Reflection: Vague tips are useless for RAG because there is no specific content to retrieve. After rewriting with named locations like MCine and Maison de The, the RAG correctly retrieved the tea section when asked 'how do I spend less at the cafe?' and the gaming section when asked about cutting Xbox spending.

Entry 9 – RAG Fallback When Server is Down



Artifact:



My hands-on-ai server keeps giving connection errors for the RAG lookup. What do I do

Python



```
def rag_lookup(question):
    from hands_on_ai.rag import get_answer
    return get_answer(question, documents=[F
```



This has no error handling so when the server is down the whole app crashes with a red Gradio error. I need it to try hands-on-ai first and if there is a connection error automatically fall back to calling the Anthropic API directly with the document as context. Can you add that fallback with a try/except?

Python



```
def rag_lookup(question):
    try:
        from hands_on_ai.rag import get_answer
        answer = get_answer(question, documents=[F
        if answer and 'Connection error' not
            return answer
    except Exception:
        pass
    print('hands-on-ai unavailable - using Anthropic API')
    prompt = (f'DOCUMENT:\n{FINANCIAL_TIPS}\n\n'
              f'QUESTION: {question}\n\nAnswer from document:')
    return call_anthropic(prompt, 'Answer from document:')
```



Context: I needed to handle the situation where the university Ollama server was unreachable from Colab so the app would still work using the Anthropic API as a backup.

My Prompt: "My hands-on-ai server keeps giving connection errors for the RAG lookup. What do I do?"

AI Response Summary: The AI returned a bare `rag_lookup` function calling `get_answer` with no error handling — so any connection failure caused a red Gradio crash with no fallback.

My Critique/Improvement: I asked for a try/except that attempts hands-on-ai first and automatically falls back to calling the Anthropic API directly with the document as context if any connection error occurs, printing a notice so the switch is visible in logs.



Result: The improved version gracefully degrades: it tries hands-on-ai, checks the result is valid, and on any failure falls back to a direct Anthropic API call with the financial tips document injected as context — the RAG tab always works regardless of server status.

Reflection: The university Ollama server at ollama.serveur.au was unreachable from my Colab session. Rather than letting the app break, adding the Anthropic fallback means the RAG tab always works regardless of server status. This is exactly what separates a prototype from a real application — graceful degradation when one component fails.

Entry 10 – Savings Calculator Logic



Artifact:



Write a savings goal calculator that takes monthly income, expenses and a goal and returns months needed

Python



```
def savings_calculator(income, expenses, goal):  
    monthly_savings = income - expenses  
    months = goal / monthly_savings  
    return f'You will reach your goal in {months} months'
```



This crashes with ZeroDivisionError if expenses equal income, and shows a confusing decimal like 7.142857 months. I also want it to show years and months separately, the weekly and daily savings amount, and the savings rate percentage. Can you add all of that with proper input validation?

Python



```
def savings_calculator(income, expenses, goal):  
    income, expenses, goal = float(income), float(expenses), float(goal)  
    if income <= 0: return 'Income must be greater than 0'  
    if expenses < 0: return 'Expenses cannot be negative'  
    if goal <= 0: return 'Goal must be greater than 0'  
    monthly = income - expenses  
    if monthly <= 0: return 'Monthly savings must be greater than 0'  
    return f'Spending ${abs(monthly):.2f} per month.  
    months = goal / monthly  
    yrs, mth = int(months // 12), int(months % 12)  
    rate = monthly / income * 100  
    weekly = monthly / 4.33  
    daily = monthly / 30  
    return (f'Time to goal: {yrs}yr {mth}mo',  
            f'Rate: {rate:.1f}% | Week: ${weekly:.2f}, Daily: ${daily:.2f}')
```





Context: I wanted to build the savings goal calculator agent tool that tells users how many months it will take to reach a target savings amount based on their income and expenses.

My Prompt: "Write a savings goal calculator that takes monthly income, expenses and a goal and returns months needed"

AI Response Summary: The AI returned a three-line function with no input validation that divides goal by monthly savings — crashing with `(ZeroDivisionError)` if income equals expenses, and displaying an unreadable decimal like 7.142857 months.

My Critique/Improvement: I asked for zero-division protection, proper input validation for all three parameters, years-and-months breakdown, weekly and daily savings figures, and the savings rate percentage — all with descriptive return strings.

Result: The improved version validates all inputs, handles the deficit case with a descriptive message, converts months to years + months, and returns weekly and daily breakdowns — giving users the most actionable numbers for day-to-day decisions.

Reflection: Testing with income=2, 500, *expenses* =1,800, goal=5, 000*gives 8 months which matches the Step 3 manual calculation exactly. The weekly breakdown of*161.66 is the most actionable number for day-to-day budgeting decisions.

Entry 11 – Category-Specific Recommendations



Artifact:

Write a function that generates spending recommendations based on category totals

Python

```
def make_recommendations(summary_df):
    for index, row in summary_df.iterrows():
        if row['Total_Spent'] > 100:
            print(f"You spent too much on {row['Category']}
```

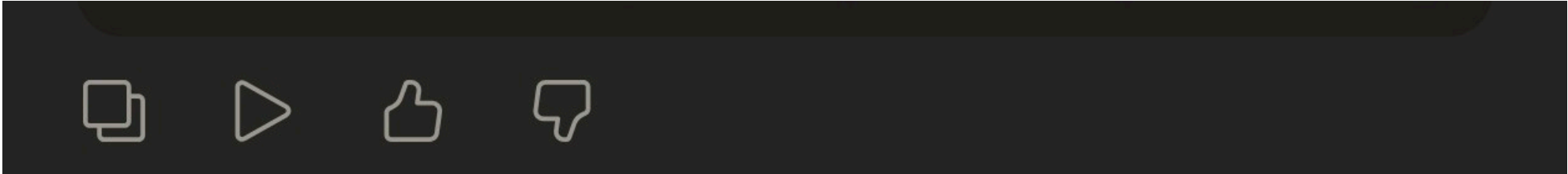


This is too generic it just prints 'you spent too much' for any category over \$100. I need specific tips per category: for Gaming mention the Xbox 48-hour wishlist rule, for Gym mention off-peak memberships, for Cinema mention MCine loyalty cards, for Tea mention Maison de The and home brewing. Thresholds should be \$50 Gaming, \$80 Gym, \$40 Cinema, \$30 Tea. And return a string not just print.

Python

```
def make_recommendations(a):
    tips = []
    for _, row in a['summary'].iterrows():
        cat, spent = row['Category'], row['Total_Spent']
        if cat == 'Gaming' and spent > 50:
            saving = round(spent * 0.4, 2)
            tips.append(f'Gaming: ${spent:.2f}. Consider the Xbox 48-hour wishlist rule. You could save ~${saving:.2f}.')
        if cat == 'Gym' and spent > 80:
            tips.append(f'Gym: ${spent:.2f}. Consider off-peak memberships or block-book PT.')
        if cat == 'Cinema' and spent > 40:
            tips.append(f'Cinema (MCine): ${spent:.2f}. Consider MCine loyalty cards. You could save ~${saving:.2f} per ticket. MCine')
        if cat == 'Tea & Cafe' and spent > 30:
            brew = round(spent * 0.85, 2)
            tips.append(f'Tea (Maison de The): ${spent:.2f}. Consider home brewing. You could save ~${brew:.2f} per cup.')
    return '\n'.join(tips) if tips else 'Spent less than $100. No recommendations.'

```

Context: I wanted a function that generates specific, actionable saving tips based on what the user actually spent — not just generic advice that could apply to anyone.

My Prompt: "Write a function that generates spending recommendations based on category totals"

AI Response Summary: The AI returned a loop that printed "You spent too much on [Category]" for any category over \$100 — generic, not actionable, and using `print` instead of returning a string usable in Gradio.

My Critique/Improvement: I asked for named tips per category (Xbox 48-hour rule for Gaming, off-peak membership for Gym, loyalty card for Cinema, home brewing for Tea), category-specific dollar thresholds (50/80/40/30), estimated savings amounts, and a returned string instead of printing.

Result: The improved version triggers specific, branded tips only above the correct threshold for each category, calculates an estimated saving, and returns a joined string that can be displayed in any Gradio textbox or included in the full report.

Reflection: Generic recommendations like 'you spent too much' add no value. The specific 48-hour rule for Xbox impulse purchases is actionable and realistic. Returning a string instead of printing means the result can be displayed in the Gradio textbox and included in the full report.

Entry 12 – Gradio Tab Layout



Artifact:

How do I create a Gradio app with multiple tabs for my finance assistant?

Python

```
import gradio as gr

with gr.Blocks() as demo:
    gr.Tab('Analysis')
    gr.Tab('Chat')
    gr.Tab('Calculator')

demo.launch()
```

This creates empty tabs with nothing in them and does not use `gr.Tabs()` as the required container. I need four tabs not three: Analysis, Chat Advisor, RAG Tips, and Savings Calculator each with its own inputs, outputs, and button wired to my existing functions. Can you show the correct structure?

Python

```
with gr.Blocks(title='Smart Finance Assistant',
               theme=gr.themes.Soft()) as demo:
    gr.Markdown('# Smart Finance Assistant')
    with gr.Tabs():
        with gr.Tab('Spending Analysis'):
            f1 = gr.File(label='Upload CSV', type='csv')
            btn1 = gr.Button('Analyse My Spending')
            out1 = gr.Textbox(label='Report')
            btn1.click(ui_analyse, inputs=f1, outputs=out1)
        with gr.Tab('Chat Advisor'):
            bot = gr.Chatbot(label='FinBot')
            msg = gr.Textbox(label='Your message')
            btn2 = gr.Button('Send', variant='primary')
            btn2.click(ui_chat, [msg, bot, f1], outputs=[bot, msg])
            # RAG and Savings tabs follow same pattern
    demo.launch(share=True)
```




Context: I needed to build the four-tab Gradio interface that ties together the CSV analysis, FinBot chat, RAG tips lookup, and savings calculator into one browser app

Entry 13 – Gradio Error Handling

My Prompt: "How do I create a Gradio app with multiple tabs for my finance assistant?"

Artifact:

AI Response Summary: The AI returned `gr.Tab('Analysis')` calls outside of any container — creating three empty, non-functional

